Report for Project 2

报告分为三个部分，分别是 *Unix Shell* 相关代码解释、`/proc/pid` 内核模块代码解释、对 *bonus* 问题的回答。

1. Unix Shell

整体思路是在主循环中持续读取用户输入，将输入解析为参数数组后调用 `fork` 创建子进程，由子进程负责执行命令，父进程则根据是否有 `&` 标志决定是否等待子进程结束。历史记录、管道、重定向均在此框架内处理。

由于基础的输入解析、`fork` 与 `exec` 调用等部分较为常规，这里只解释各核心功能的关键实现。

历史记录（`!!`）：每次成功执行命令前将输入存入 `history` 字符数组；当检测到输入为 `!!` 时，直接将 `history` 复制回 `input` 继续执行，并用 `has_history` 标志防止在无历史时调用。

后台执行（`&`）：在 `fork` 之前遍历参数数组末尾，若最后一个参数为 `&` 则将其置 `NULL` 并设 `background = 1`，父进程据此跳过 `waitpid`。

管道：在子进程中检测参数里是否存在 `|`，若存在则再次 `fork` 出孙进程。核心代码如下：

```
args[pipe_idx] = NULL;
if (fork() == 0) {
    dup2(fd[1], STDOUT_FILENO);
    // ...
    execvp(args[0], args);
}
dup2(fd[0], STDIN_FILENO);
execvp(args[pipe_idx + 1], &args[pipe_idx + 1]);
```

将管道写端重定向到孙进程的标准输出，将读端重定向到子进程的标准输入，两侧分别执行 `|` 前后的命令，实现进程间通信。

I/O 重定向：遍历参数数组，检测到 `>` 或 `<` 后用 `open` 打开目标文件，再通过 `dup2` 将文件描述符覆盖标准输出或标准输入，并将 `>` / `<` 置 `NULL` 以避免传入 `execvp`。

2. `/proc/pid` 内核模块

整体思路是通过 `proc_ops` 注册 `proc_read` 和 `proc_write` 两个回调，写操作将用户传入的 PID 字符串转换并存入内核全局变量 `last_pid`，读操作则根据 `last_pid` 查询对应进程信息并返回给用户态。

写操作：核心在于安全地从用户空间获取数据并转换类型。

```
k_mem = kmalloc(count + 1, GFP_KERNEL);
if (copy_from_user(k_mem, usr_buf, count)) { ... }
kstrtoint(k_mem, 10, &last_pid);
kfree(k_mem);
```

先在内核空间分配缓冲区，用 `copy_from_user` 将用户态字符串安全地复制进来，再用 `kstrtoint` 解析为整数存入 `last_pid`，最后释放内存。

读操作：核心是通过 PID 反查 `task_struct`。

```
pid_struct = find_vpid(last_pid);
task = pid_task(pid_struct, PIDTYPE_PID);
rv = sprintf(buffer, "command = [%s] pid = [%d] state = [%u]\n",
             task->comm, task->pid, task->__state);
copy_to_user(usr_buf, buffer, rv);
```

`find_vpid` 将整数 PID 转换为内核 `pid` 结构体，`pid_task` 进一步获取对应的 `task_struct`，从中提取命令名 `comm`、PID 及状态字段，格式化后通过 `copy_to_user` 写回用户空间。此外，利用 `*pos > 0` 的判断保证 `cat` 命令只触发一次有效读取。

3. Bonus：匿名管道与命名管道的异同

本题 Unix Shell 中父子进程使用**匿名管道**（anonymous pipe）通信，Linux 系统中还广泛使用**命名管道**（named pipe / FIFO）。二者的核心差异如下：

1. **生命周期与标识**。 匿名管道通过 `pipe(fd)` 创建，仅存在于内存中，没有文件系统路径，随进程关闭而消失；命名管道通过 `mkfifo` 创建，在文件系统中有实际路径（如 `/tmp/myfifo`），独立于进程生命周期而持久存在。

2. **使用范围**。 匿名管道只能用于具有亲缘关系的进程（如父子、兄弟进程）之间，因为文件描述符只能通过 `fork` 继承；命名管道可在任意无亲缘关系的进程间通信，只需双方以路径打开同一 FIFO 文件即可。

3. **通信模型**。 二者均为半双工、基于字节流的单向通信，读写均会阻塞直到对端就绪，本质上共享同一套内核管道机制，差异仅在于寻址方式不同。